using a fluent API and operators to process the data received asynchronously.

Besides the standard operators available to Java streams, you can use more powerful ones to apply backpressure, handle errors, and increase application resilience. For example, you'll see how to use the `retryWhen()` and `timeout()` operators to make the interaction between Order Service and Catalog Service more robust. Operators can perform actions on a publisher and return a new publisher without modifying the original one, so you can build functional and immutable data streams with ease.

Project Reactor is the foundation of the Spring reactive stack, which lets you implement your business logic in terms of `Mono<T>` and `Flux<T>`. In the next section, you'll learn more about which options you have for building reactive applications with Spring.

### *8.1.3 Understanding the Spring reactive stack*

When you build applications with Spring, you can choose between a servlet stack and a reactive stack. The servlet stack relies on synchronous, blocking I/O and uses the thread-per-request model to handle requests. On the other hand, the reactive stack relies on asynchronous, non-blocking I/O and uses the event loop model to handle requests.

The servlet stack is based on the Servlet API and a Servlet container (such as Tomcat). In contrast, the reactive model is based on the Reactive Streams API (implemented by Project Reactor) and either Netty or a Servlet container (version 3.1 as a minimum). Both stacks let you build RESTful applications using either classes annotated as `@RestController` (which you used in chapter 3) or functional endpoints called router functions (which you'll learn about in chapter 9). The servlet stack uses Spring MVC,